```
# ------------------------------------------------------------------------
#
#    11111111      11      11    11      11               1111
#  1111    1111    11      11    11      11             111111
#  1111            11      11    11      11               1111
#    11111111      11      11    1111111111    ==         1111
#          1111     11    11     11      11               1111
#          1111    11    11      11      11               1111
#  1111    1111      11  11      11      11               1111
#    11111111          1111      11      11          1111111111
#
#
#                        VIRGINIA HYDRODYNAMICS #1
#
# ------------------------------------------------------------------------
# Julia port of vhone.f90
# Main VH-1 driver loop for the julia_fvh1_12 code (parallel sweeps + mc3d).
cd(@__DIR__)

include("vh1state.jl")
include("parabola.jl")
include("boundary.jl")
include("states.jl")
include("evolve.jl")
include("remap.jl")
include("ppmlr.jl")
include("init.jl")
include("stress.jl")
include("prin.jl")
include("dtcon.jl")
include("mc3d.jl")
include("sweepx.jl")
include("sweepy.jl")
include("sweepz.jl")

using Dates
using Random

const g = globalvars()
const z = zone()
const zg = zonegrid()
const zo = zoneobs()
const sweep_pool = SweepThreadPool()
const mc_pool = McThreadPool()

function parse_indat()
    prefix_val = ""
    endtime_val = 0.0
    tprin_val = 0.0

    if isfile("indat")
        for line in eachline("indat")
            line = strip(line)
            if isempty(line) || startswith(line, "!") || startswith(line, "&") |
| startswith(line, "/")
                continue
            end
            for part in split(line, ',')
                kv = split(part, '=')
                if length(kv) == 2
                    key = strip(lowercase(kv[1]))
                    value = strip(kv[2])
                    value = strip(replace(value, r"/\s*$" => ""))
                    if key == "prefix"
                        prefix_val = strip(value, ['\'', '"'])
                    elseif key == "endtime"
```

```
                        endtime_val = parse(Float64, value)
                    elseif key == "tprin"
                        tprin_val = parse(Float64, value)
                    end
                end
            end
        end
    end

    return prefix_val, endtime_val, tprin_val
end

function open_output_files()
    return Dict{Int,IO}(
        65 => open("rho_x.dat", "w"),
        66 => open("rho_y.dat", "w"),
        67 => open("rho_z.dat", "w"),
        68 => open("vel_x.dat", "w"),
        88 => open("vel_y.dat", "w"),
        98 => open("vel_z.dat", "w"),
        69 => open("temp_tran.dat", "w"),
        70 => open("pixx_tran.dat", "w"),
        71 => open("s_x.dat", "w"),
        75 => open("energy.dat", "w"),
        76 => open("radii.dat", "w"),
        78 => open("entropy.dat", "w"),
        79 => open("momentum.dat", "w"),
        80 => open("pipi.dat", "w"),
        81 => open("pipiim.dat", "w"),
        82 => open("pi-spec.dat", "w"),
        83 => open("rhokx.dat", "w")
    )
end

function close_output_files(io_units::Dict{Int,<:IO})
    for io in values(io_units)
        close(io)
    end
    return nothing
end

function format_write(io::IO, args...)
    println(io, join(args, " "))
    return nothing
end

function vhone!()
    Random.seed!(123456)
    g.ntrial = 0
    g.nacc = 0

    prefix, g.endtime, g.tprin = parse_indat()

    if max(imax, jmax, kmax) + 12 > maxsweep
        error("maxsweep too small")
    end
    if ndim != 3
        error("This is a 3d code")
    end

    io_units = open_output_files()
    history_name = string(prefix, "hst")
    history_io = open(history_name, "w")
    println(history_io, "History File for VH-1 simulation run on ", Dates.format
(now(), "mm/dd/yyyy"))
    println(history_io)
```

```
    init!(g, z, zg)
    myprin!(io_units, g, z, zg)

    nprin = floor(Int, (g.tprin + 0.001) / g.dt)
    icor = 40
    iequ = 1
    nk = 4
    iconfmax = 5000

    if icor > Int(g.endtime / 2.0 / g.dt) + iequ
        error("icor too large")
    end

    zo.spec .= 0.0
    zo.spec2 .= 0.0

    iconf = 0
    ispec = 0
    ncycend = 100000

    Arepixky = zeros(Float64, iconfmax, nk)
    Aimpixky = zeros(Float64, iconfmax, nk)
    cor = zeros(Float64, icor + 1, nk)
    cori = zeros(Float64, icor + 1, nk)
    cor2 = zeros(Float64, icor + 1, nk)
    cori2 = zeros(Float64, icor + 1, nk)

    while g.ncycle < ncycend
        iconf += 1
        g.ncycle += 2
        g.ncycp += 2
        println("conf = ", iconf, " t = ", g.time, " dt = ", g.dt)

        etot, ekin, eint = energy!(z, zg, 0.0, 0.0, 0.0)
        stot, rtot = entropy!(z, zg, 0.0, 0.0)
        rx, ry, rz = scalesize!(z, zg, 0.0, 0.0, 0.0)

        ptot = zeros(Float64, 3)
        pdipole = zeros(Float64, 3, 3)
        mass, ptot, pdipole = momentum!(z, zg, 0.0, ptot, pdipole)
        pixky!(z, zo)

        ispec += 1
        for j in 1:jmax
            zo.spec[j] += zo.repixky[j]^2 + zo.impixky[j]^2
            zo.spec2[j] += (zo.repixky[j]^2 + zo.impixky[j]^2)^2
        end
        for k in 1:nk
            Arepixky[iconf, k] = zo.repixky[k]
            Aimpixky[iconf, k] = zo.impixky[k]
        end

        format_write(io_units[75], g.time, etot, ekin, eint)
        format_write(io_units[76], g.time, rx, ry, rz)
        format_write(io_units[78], g.time, stot, rtot, stot / rtot)
        format_write(io_units[79], g.time, mass, ptot[1], pdipole[1, 2] / mass,
zo.impixky[1])
        format_write(io_units[83], g.time, zo.rhokx[1], zo.rhokx[2], zo.rhokx[3]
)

        sweepx!(sweep_pool, g, z, zg)
        sweepy!(sweep_pool, g, z, zg)
        sweepz!(sweep_pool, g, z, zg)

        g.time += g.dt
```

```
        g.timep += g.dt

        sweepz!(sweep_pool, g, z, zg)
        sweepy!(sweep_pool, g, z, zg)
        sweepx!(sweep_pool, g, z, zg)

        g.time += g.dt
        g.timep += g.dt

        vcon!(g, z, zg)
        g.dt = g.dtfix

        mc3d!(g, z, mc_pool)
        mc3d!(g, z, mc_pool)

        vcon!(g, z, zg)

        if g.ncycp >= nprin
            myprin!(io_units, g, z, zg)
            g.ncycp = 0
        end

        if g.time > g.endtime
            g.ncycle = ncycend
        end
    end

    isample = 0
    for i in iequ:(iconf - icor)
        isample += 1
        for j in 0:icor
            for k in 1:nk
                recor = Arepixky[i, k] * Arepixky[i + j, k] + Aimpixky[i, k] * A
impixky[i + j, k]
                imcor = Arepixky[i, k] * Aimpixky[i + j, k] - Aimpixky[i, k] * A
repixky[i + j, k]
                cor[j + 1, k] += recor
                cori[j + 1, k] += imcor
                cor2[j + 1, k] += recor^2
                cori2[j + 1, k] += imcor^2
            end
        end
    end

    for j in 0:icor
        for k in 1:nk
            cor[j+1,k]  = cor[j+1,k]/float(isample)
            cori[j+1,k] = cori[j+1,k]/float(isample)
            cor2[j+1,k] = cor2[j+1,k]/float(isample)
            cori2[j+1,k] = cori2[j+1,k]/float(isample)
        end
        cork1 = cor[j + 1, 1] / cor[1, 1]
        cork2 = cor[j + 1, 2] / cor[1, 2]
        cork3 = cor[j + 1, 3] / cor[1, 3]
        dc1 = sqrt((cor2[j + 1, 1] - cor[j + 1, 1]^2) / isample) / cor[1, 1]
        dc2 = sqrt((cor2[j + 1, 2] - cor[j + 1, 2]^2) / isample) / cor[1, 2]
        dc3 = sqrt((cor2[j + 1, 3] - cor[j + 1, 3]^2) / isample) / cor[1, 3]
        format_write(io_units[80], 2.0 * g.dt * j, cork1, dc1, cork2, dc2, cork3
, dc3)

        cork1 = cori[j + 1, 1] / cor[1, 1]
        cork2 = cori[j + 1, 2] / cor[1, 2]
        cork3 = cori[j + 1, 3] / cor[1, 3]
        dc1 = sqrt((cori2[j + 1, 1] - cori[j + 1, 1]^2) / isample) / cor[1, 1]
        dc2 = sqrt((cori2[j + 1, 2] - cori[j + 1, 2]^2) / isample) / cor[1, 2]
        dc3 = sqrt((cori2[j + 1, 3] - cori[j + 1, 3]^2) / isample) / cor[1, 3]
```

```
        format_write(io_units[81], 2.0 * g.dt * j, cork1, dc1, cork2, dc2, cork3
, dc3)
    end

    for j in 1:jmax
        zo.spec[j] /= ispec
        zo.spec2[j] /= ispec
        format_write(io_units[82], j, zo.spec[j], sqrt((zo.spec2[j] - zo.spec[j]
^2) / ispec))
    end

    println("acceptance rate ", Float64(g.nacc) / Float64(g.ntrial))

    close(history_io)
    close_output_files(io_units)

    return nothing
end
```

```
# Julia port of init.f90

function grid!(nzones::Integer, xmin::Real, xmax::Real, xa::Vector{Float64}, xc:
::Vector{Float64}, dx::Vector{Float64})
    dxfac = (xmax - xmin) / Float64(nzones)
    for n in 1:nzones
        xa[n] = xmin + (n - 1) * dxfac
        dx[n] = dxfac
        xc[n] = xa[n] + 0.5 * dx[n]
    end
    return nothing
end

function init!(g::globalvars, z::zone, zg::zonegrid)
    zg.nleftx = 3
    zg.nrightx = 3
    zg.nlefty = 3
    zg.nrighty = 3
    zg.nleftz = 3
    zg.nrightz = 3

    xmin = -5.0
    xmax = 5.0
    ymin = -5.0
    ymax = 5.0
    zmin = -5.0
    zmax = 5.0

    g.time = 0.0
    g.timep = 0.0
    g.ncycle = 0
    g.ncycp = 0

    grid!(imax, xmin, xmax, zg.zxa, zg.zxc, zg.zdx)
    grid!(jmax, ymin, ymax, zg.zya, zg.zyc, zg.zdy)
    grid!(kmax, zmin, zmax, zg.zza, zg.zzc, zg.zdz)

    g.dtfix = 0.05

    lambda = 1.0
    e0ef = 1.0
    n00 = 1.0

    for k in 1:kmax
        for j in 1:jmax
            for i in 1:imax
                rr2 = zg.zxa[i]^2 + zg.zya[j]^2 + lambda^2 * zg.zza[k]^2
                vconf = rr2 / 1.0
                rhoi = nav * exp(-vconf)
                z.zro[i, j, k] = max(rhoi, smallr)
                z.zpr[i, j, k] = z.zro[i, j, k] * tav
                z.zux[i, j, k] = 0.0
                z.zuy[i, j, k] = 0.0
                z.zuz[i, j, k] = 0.0
            end
        end
    end

    for k in 1:kmax
        for j in 1:jmax
            for i in 1:imax
                z.zro[i, j, k] = nav
                rr2 = zg.zxa[i]^2 + zg.zya[j]^2 + zg.zza[k]^2
                z.zro[i, j, k] = nav * (1.0 + 0.0 * exp(-rr2))
                z.zpr[i, j, k] = z.zro[i, j, k] * tav
                z.zux[i, j, k] = 0.0
```

```
                z.zuy[i, j, k] = 0.0
                z.zuz[i, j, k] = 0.0
            end
        end
    end

    v0shear = 0.5
    for k in 1:kmax
        for j in 1:jmax
            for i in 1:imax
               z.zro[i, j, k] = nav
               rr2 = zg.zxa[i]^2 + zg.zya[j]^2 + zg.zza[k]^2
               z.zpr[i, j, k] = z.zro[i, j, k] * tav
               z.zux[i, j, k] = v0shear * sin(pi * zg.zya[j] / ymax)
               z.zuy[i, j, k] = 0.0
               z.zuz[i, j, k] = 0.0
            end
        end
    end

    for k in 1:kmax
        for j in 1:jmax
            for i in 1:imax
                z.zro[i, j, k] = nav
                z.zro[i, j, k] = nav * (1.0 + 0.1 * sin(pi * zg.zxa[i] / xmax))
                z.zpr[i, j, k] = nav * tav * (z.zro[i, j, k] / nav)^gam
                z.zux[i, j, k] = 0.0
                z.zuy[i, j, k] = 0.0
                z.zuz[i, j, k] = 0.0
            end
        end
    end

    g.dt = g.dtfix
    return nothing
end
```

```
# Julia port of dtcon.f90

"""
    vcon!(g::globalvars, z::zone, zg::zonegrid)

Clamp the velocity components in the zone arrays to the CFL bounds implied by dt
.
"""
function vcon!(g::globalvars, z::zone, zg::zonegrid)
    vxbound = zg.zdx[1] / g.dt * 0.45
    vybound = zg.zdy[1] / g.dt * 0.45
    vzbound = zg.zdz[1] / g.dt * 0.45

    for k in 1:kmax
        for j in 1:jmax
            for i in 1:imax
                xvel = abs(z.zux[i, j, k])
                yvel = abs(z.zuy[i, j, k])
                zvel = abs(z.zuz[i, j, k])

                if xvel > vxbound
                    z.zux[i, j, k] *= vxbound / xvel
                end
                if yvel > vybound
                    z.zuy[i, j, k] *= vybound / yvel
                end
                if zvel > vzbound
                    z.zuz[i, j, k] *= vzbound / zvel
                end
            end
        end
    end

    return nothing
end
```

```
# Julia port of stress.f90

function energy!(z::zone, zg::zonegrid, etot::Real, ekin::Real, eint::Real)
    ekin = 0.0
    eint = 0.0

    for i in 1:imax
        for j in 1:jmax
            for k in 1:kmax
                d3x = zg.zdx[i] * zg.zdy[j] * zg.zdz[k]
                v2 = z.zux[i, j, k]^2 + z.zuy[i, j, k]^2 + z.zuz[i, j, k]^2
                ekin += 0.5 * z.zro[i, j, k] * v2 * d3x
                eint += z.zpr[i, j, k] / gamm * d3x
            end
        end
    end

    etot = ekin + eint
    return etot, ekin, eint
end

function scalesize!(z::zone, zg::zonegrid, rx::Real, ry::Real, rz::Real)
    xmtot = 0.0
    rx2 = 0.0
    ry2 = 0.0
    rz2 = 0.0

    for i in 1:imax
        for j in 1:jmax
            for k in 1:kmax
                xmtot += z.zro[i, j, k]
                rx2 += z.zro[i, j, k] * zg.zxa[i]^2
                ry2 += z.zro[i, j, k] * zg.zya[j]^2
                rz2 += z.zro[i, j, k] * zg.zza[k]^2
            end
        end
    end

    rx = sqrt(rx2 / xmtot) * sqrt(2.0)
    ry = sqrt(ry2 / xmtot) * sqrt(2.0)
    rz = sqrt(rz2 / xmtot) * sqrt(2.0)
    return rx, ry, rz
end

function entropy!(z::zone, zg::zonegrid, stot::Real, rtot::Real)
    const_ = 400.0
    stot_sum = 0.0
    rtot_sum = 0.0

    for i in 1:imax
        for j in 1:jmax
            for k in 1:kmax
                d3x = zg.zdx[i] * zg.zdy[j] * zg.zdz[k]
                rloc = z.zro[i, j, k]
                ploc = z.zpr[i, j, k]
                sloc = rloc * (2.5 + log(const_ * ploc^1.5 / rloc^2.5))
                rtot_sum += rloc * d3x
                stot_sum += sloc * d3x
            end
        end
    end

    stot = stot_sum
    rtot = rtot_sum
    return stot, rtot
end
```

```
function slocal(rloc::Real, ploc::Real)
    const_ = 400.0
    return rloc * (2.5 + log(const_ * ploc^1.5 / rloc^2.5))
end

function momentum!(z::zone, zg::zonegrid, mass::Real, ptot::Vector{Float64}, pdi
pole::Matrix{Float64})
    mass = 0.0
    fill!(ptot, 0.0)
    fill!(pdipole, 0.0)

    for i in 1:imax
        for j in 1:jmax
            for k in 1:kmax
                d3x = zg.zdx[i] * zg.zdy[j] * zg.zdz[k]
                mass += d3x * z.zro[i, j, k]
                ptot[1] += d3x * z.zro[i, j, k] * z.zux[i, j, k]
                ptot[2] += d3x * z.zro[i, j, k] * z.zuy[i, j, k]
                ptot[3] += d3x * z.zro[i, j, k] * z.zuz[i, j, k]
                pdipole[1, 1] += d3x * z.zro[i, j, k] * z.zux[i, j, k] * zg.zxa[
i]
                pdipole[1, 2] += d3x * z.zro[i, j, k] * z.zux[i, j, k] * zg.zya[
j]
                pdipole[1, 3] += d3x * z.zro[i, j, k] * z.zux[i, j, k] * zg.zza[
k]
                pdipole[2, 1] += d3x * z.zro[i, j, k] * z.zuy[i, j, k] * zg.zxa[
i]
                pdipole[2, 2] += d3x * z.zro[i, j, k] * z.zuy[i, j, k] * zg.zya[
j]
                pdipole[2, 3] += d3x * z.zro[i, j, k] * z.zuy[i, j, k] * zg.zza[
k]
                pdipole[3, 1] += d3x * z.zro[i, j, k] * z.zuz[i, j, k] * zg.zxa[
i]
                pdipole[3, 2] += d3x * z.zro[i, j, k] * z.zuz[i, j, k] * zg.zya[
j]
                pdipole[3, 3] += d3x * z.zro[i, j, k] * z.zuz[i, j, k] * zg.zza[
k]
            end
        end
    end

    return mass, ptot, pdipole
end

function pixky!(z::zone, zo::zoneobs)
    nk = jmax

    Threads.@threads for in_ in 1:nk
        rep = 0.0
        imp = 0.0
        rh = 0.0
        @inbounds for k in 1:kmax, j in 1:jmax, i in 1:imax
            rho = z.zro[i, j, k]
            pix = rho * z.zux[i, j, k]
            rep += pix * PIXKY_COS_J[j, in_]
            imp += pix * PIXKY_SIN_J[j, in_]
            rh += rho * PIXKY_SIN_I[i, in_]
        end
        zo.repixky[in_] = rep * PIXKY_XNORM
        zo.impixky[in_] = imp * PIXKY_XNORM
        zo.rhokx[in_] = rh * PIXKY_XNORM
    end

    return nothing
end
```

```
# Julia port of sweepx.f90 âM-^@M-^T parallel over (j, k) lines via Threads.@thr
eads

function sweepx!(pool::SweepThreadPool, g::globalvars, z::zone, zg::zonegrid)
    nleft = zg.nleftx
    nright = zg.nrightx
    nmin = 7
    nmax = imax + 6

    Threads.@threads for jj in 1:(jmax * kmax)
        j = (jj - 1) Ã• kmax + 1
        k = (jj - 1) % kmax + 1
        tid = Threads.threadid()
        sweeps = pool.sweeps[tid]
        w = pool.work[tid]

        sweeps.nleft = nleft
        sweeps.nright = nright
        sweeps.nmin = nmin
        sweeps.nmax = nmax

        @inbounds for i in 1:imax
            n = i + 6

            sweeps.r[n] = z.zro[i, j, k]
            sweeps.p[n] = z.zpr[i, j, k]
            sweeps.u[n] = z.zux[i, j, k]
            sweeps.v[n] = z.zuy[i, j, k]
            sweeps.w[n] = z.zuz[i, j, k]

            sweeps.xa0[n] = zg.zxa[i]
            sweeps.dx0[n] = zg.zdx[i]
            sweeps.xa[n] = zg.zxa[i]
            sweeps.dx[n] = zg.zdx[i]

            sweeps.p[n] = max(smallp, sweeps.p[n])
            sweeps.e[n] = sweeps.p[n] / (sweeps.r[n] * gamm) +
                0.5 * (sweeps.u[n]^2 + sweeps.v[n]^2 + sweeps.w[n]^2)
        end

        ppmlr!(sweeps, g, w)

        @inbounds for i in 1:imax
            n = i + 6
            z.zro[i, j, k] = sweeps.r[n]
            z.zpr[i, j, k] = sweeps.p[n]
            z.zux[i, j, k] = sweeps.u[n]
            z.zuy[i, j, k] = sweeps.v[n]
            z.zuz[i, j, k] = sweeps.w[n]
        end
    end

    return nothing
end
```

```
# Julia port of sweepy.f90 âM-^@M-^T parallel over (i, k) lines via Threads.@thr
eads

function sweepy!(pool::SweepThreadPool, g::globalvars, z::zone, zg::zonegrid)
    nleft = zg.nlefty
    nright = zg.nrighty
    nmin = 7
    nmax = jmax + 6

    Threads.@threads for jj in 1:(imax * kmax)
        i = (jj - 1) Ã• kmax + 1
        k = (jj - 1) % kmax + 1
        tid = Threads.threadid()
        sweeps = pool.sweeps[tid]
        w = pool.work[tid]

        sweeps.nleft = nleft
        sweeps.nright = nright
        sweeps.nmin = nmin
        sweeps.nmax = nmax

        @inbounds for j in 1:jmax
            n = j + 6

            sweeps.r[n] = z.zro[i, j, k]
            sweeps.p[n] = z.zpr[i, j, k]
            sweeps.u[n] = z.zuy[i, j, k]
            sweeps.v[n] = z.zuz[i, j, k]
            sweeps.w[n] = z.zux[i, j, k]

            sweeps.xa0[n] = zg.zya[j]
            sweeps.dx0[n] = zg.zdy[j]
            sweeps.xa[n] = zg.zya[j]
            sweeps.dx[n] = zg.zdy[j]

            sweeps.p[n] = max(smallp, sweeps.p[n])
            sweeps.e[n] = sweeps.p[n] / (sweeps.r[n] * gamm) +
                0.5 * (sweeps.u[n]^2 + sweeps.v[n]^2 + sweeps.w[n]^2)
        end

        ppmlr!(sweeps, g, w)

        @inbounds for j in 1:jmax
            n = j + 6
            z.zro[i, j, k] = sweeps.r[n]
            z.zpr[i, j, k] = sweeps.p[n]
            z.zuy[i, j, k] = sweeps.u[n]
            z.zuz[i, j, k] = sweeps.v[n]
            z.zux[i, j, k] = sweeps.w[n]
        end
    end

    return nothing
end
```

```
# Julia port of sweepz.f90 âM-^@M-^T parallel over (i, j) lines via Threads.@thr
eads

function sweepz!(pool::SweepThreadPool, g::globalvars, z::zone, zg::zonegrid)
    nleft = zg.nleftz
    nright = zg.nrightz
    nmin = 7
    nmax = kmax + 6

    Threads.@threads for jj in 1:(imax * jmax)
        i = (jj - 1) Ã• jmax + 1
        j = (jj - 1) % jmax + 1
        tid = Threads.threadid()
        sweeps = pool.sweeps[tid]
        w = pool.work[tid]

        sweeps.nleft = nleft
        sweeps.nright = nright
        sweeps.nmin = nmin
        sweeps.nmax = nmax

        @inbounds for k in 1:kmax
            n = k + 6

            sweeps.r[n] = z.zro[i, j, k]
            sweeps.p[n] = z.zpr[i, j, k]
            sweeps.u[n] = z.zuz[i, j, k]
            sweeps.v[n] = z.zux[i, j, k]
            sweeps.w[n] = z.zuy[i, j, k]

            sweeps.xa0[n] = zg.zza[k]
            sweeps.dx0[n] = zg.zdz[k]
            sweeps.xa[n] = zg.zza[k]
            sweeps.dx[n] = zg.zdz[k]

            sweeps.p[n] = max(smallp, sweeps.p[n])
            sweeps.e[n] = sweeps.p[n] / (sweeps.r[n] * gamm) +
                0.5 * (sweeps.u[n]^2 + sweeps.v[n]^2 + sweeps.w[n]^2)
        end

        ppmlr!(sweeps, g, w)

        @inbounds for k in 1:kmax
            n = k + 6
            z.zro[i, j, k] = sweeps.r[n]
            z.zpr[i, j, k] = sweeps.p[n]
            z.zuz[i, j, k] = sweeps.u[n]
            z.zux[i, j, k] = sweeps.v[n]
            z.zuy[i, j, k] = sweeps.w[n]
        end
    end

    return nothing
end
```

```
# Julia port of mc3d.f90 âM-^@M-^T 8-color checkerboard parallelization.

@inline function gaussian_random(rng::Random.AbstractRNG)
    r1 = rand(rng)
    r2 = rand(rng)
    if r1 <= 0.0
        r1 = 1.0e-18
    end
    return sqrt(-2.0 * log(r1)) * cos(2.0 * ÏM-^@ * r2)
end

@inline function mc_corner_update!(dhtot::Float64, qnew_cube, pcube1, pcube2, pc
ube3,
                                   p1, p2, p3, q, zro,
                                   ci::Int, cj::Int, ck::Int,
                                   si::Float64, sj::Float64, sk::Float64,
                                   dp11, dp12, dp13, dp21, dp22, dp23, dp31, dp3
2, dp33,
                                   cube_idx::Int)
    p_old1 = p1[ci, cj, ck]
    p_old2 = p2[ci, cj, ck]
    p_old3 = p3[ci, cj, ck]

    p_new1 = p_old1 + 0.25 * (si * dp11 + sj * dp12 + sk * dp13)
    p_new2 = p_old2 + 0.25 * (si * dp21 + sj * dp22 + sk * dp23)
    p_new3 = p_old3 + 0.25 * (si * dp31 + sj * dp32 + sk * dp33)

    pcube1[cube_idx] = p_new1
    pcube2[cube_idx] = p_new2
    pcube3[cube_idx] = p_new3

    mass = V0 * zro[ci, cj, ck]
    H_old = (p_old1^2 + p_old2^2 + p_old3^2) / (2.0 * mass)
    H_new = (p_new1^2 + p_new2^2 + p_new3^2) / (2.0 * mass)

    dhtot += H_new - H_old
    qnew_cube[cube_idx] = q[ci, cj, ck] - (H_new - H_old) / mass
    return dhtot
end

@inline function mc3d_cell!(rng::Random.AbstractRNG,
                            p1, p2, p3, q, zro,
                            sc::McScratch,
                            i::Int, j::Int, k::Int,
                            factor::Float64)
    lambda = sc.lambda
    lambda_t = sc.lambda_t
    delta_p = sc.delta_p
    qnew_cube = sc.qnew_cube
    pcube1 = sc.pcube1
    pcube2 = sc.pcube2
    pcube3 = sc.pcube3

    i_corners = (mod(i - 1, imax) + 1, mod(i, imax) + 1)
    j_corners = (mod(j - 1, jmax) + 1, mod(j, jmax) + 1)
    k_corners = (mod(k - 1, kmax) + 1, mod(k, kmax) + 1)

    @inbounds for col in 1:3, row in 1:3
        lambda[row, col] = gaussian_random(rng)
    end

    trace_lambda = lambda[1, 1] + lambda[2, 2] + lambda[3, 3]
    @inbounds for col in 1:3, row in 1:3
        lambda_t[row, col] = lambda[row, col] + lambda[col, row]
    end
    lambda_t[1, 1] -= 2.0 / 3.0 * trace_lambda
```

```
    lambda_t[2, 2] -= 2.0 / 3.0 * trace_lambda
    lambda_t[3, 3] -= 2.0 / 3.0 * trace_lambda
    @inbounds for col in 1:3, row in 1:3
        delta_p[row, col] = factor * lambda_t[row, col]
    end

    dp11 = delta_p[1, 1]
    dp12 = delta_p[1, 2]
    dp13 = delta_p[1, 3]
    dp21 = delta_p[2, 1]
    dp22 = delta_p[2, 2]
    dp23 = delta_p[2, 3]
    dp31 = delta_p[3, 1]
    dp32 = delta_p[3, 2]
    dp33 = delta_p[3, 3]

    dhtot = 0.0
    @inbounds for cube_idx in 1:8
        ci = i_corners[MC_I_SLOT[cube_idx]]
        cj = j_corners[MC_J_SLOT[cube_idx]]
        ck = k_corners[MC_K_SLOT[cube_idx]]
        dhtot = mc_corner_update!(
            dhtot, qnew_cube, pcube1, pcube2, pcube3,
            p1, p2, p3, q, zro,
            ci, cj, ck,
            MC_SIGN_I[cube_idx], MC_SIGN_J[cube_idx], MC_SIGN_K[cube_idx],
            dp11, dp12, dp13, dp21, dp22, dp23, dp31, dp32, dp33,
            cube_idx
        )
    end

    accept = false
    if rand(rng) < exp(-dhtot / tav)
        accept = true
        @inbounds for cube_idx in 1:8
            if qnew_cube[cube_idx] <= 0.0
                accept = false
                break
            end
        end
    end

    if accept
        @inbounds for cube_idx in 1:8
            ci = i_corners[MC_I_SLOT[cube_idx]]
            cj = j_corners[MC_J_SLOT[cube_idx]]
            ck = k_corners[MC_K_SLOT[cube_idx]]
            p1[ci, cj, ck] = pcube1[cube_idx]
            p2[ci, cj, ck] = pcube2[cube_idx]
            p3[ci, cj, ck] = pcube3[cube_idx]
            q[ci, cj, ck] = qnew_cube[cube_idx]
        end
        return true
    end
    return false
end

function mc3d!(g::globalvars, z::zone, pool::McThreadPool)
    mc = pool.grid
    p1 = mc.p1
    p2 = mc.p2
    p3 = mc.p3
    q = mc.q

    p1 .= V0 .* z.zro .* z.zux
    p2 .= V0 .* z.zro .* z.zuy
```

```
    p3 .= V0 .* z.zro .* z.zuz
    q  .= z.zpr ./ (gamm .* z.zro)

    factor = A0 * sqrt(etaav * tav * g.dt / V0)

    ntrial_add = zeros(Int, Threads.maxthreadid())
    nacc_add = zeros(Int, Threads.maxthreadid())

    # 8 phases: cells with (i%2, j%2, k%2) == (pi, pj, pk).
    # Same-phase cells are at least Chebyshev distance 2 apart, so corner
    # writes do not overlap within a phase.
    @inbounds for phase in 0:7
        pi = phase & 1
        pj = (phase >> 1) & 1
        pk = (phase >> 2) & 1

        ni = mc_phase_count(imax, pi)
        nj = mc_phase_count(jmax, pj)
        nk = mc_phase_count(kmax, pk)
        ncells = ni * nj * nk

        Threads.@threads for idx in 1:ncells
            tid = Threads.threadid()
            sc = pool.scratch[tid]
            rng = sc.rng

            tmp = idx - 1
            ik = tmp % nk + 1
            tmp = tmp Ã• nk
            ij = tmp % nj + 1
            ii = tmp Ã• nj + 1

            i = 2 * (ii - 1) + (pi == 0 ? 2 : 1)
            j = 2 * (ij - 1) + (pj == 0 ? 2 : 1)
            k = 2 * (ik - 1) + (pk == 0 ? 2 : 1)

            ntrial_add[tid] += 1
            if mc3d_cell!(rng, p1, p2, p3, q, z.zro, sc, i, j, k, factor)
                nacc_add[tid] += 1
            end
        end
    end

    g.ntrial += sum(ntrial_add)
    g.nacc += sum(nacc_add)

    inv_v0 = 1.0 / V0
    @inbounds for k in 1:kmax, j in 1:jmax, i in 1:imax
        rho_inv = inv_v0 / z.zro[i, j, k]
        z.zux[i, j, k] = p1[i, j, k] * rho_inv
        z.zuy[i, j, k] = p2[i, j, k] * rho_inv
        z.zuz[i, j, k] = p3[i, j, k] * rho_inv
        z.zpr[i, j, k] = gamm * q[i, j, k] * z.zro[i, j, k]
    end

    return nothing
end
```

```
# Julia port of prin.f90

function myprin!(io_units::Dict{Int,<:IO}, g::globalvars, z::zone, zg::zonegrid)
    center_j = jmax Ã• 2 + 1
    center_k = kmax Ã• 2 + 1
    center_i = imax Ã• 2 + 1

    for i in 1:imax
        println(io_units[65], g.time, " ", zg.zxa[i], " ", z.zro[i, center_j, ce
nter_k])
    end

    for j in 1:jmax
        println(io_units[66], g.time, " ", zg.zya[j], " ", z.zro[center_i, j, ce
nter_k])
    end

    for k in 1:kmax
        println(io_units[67], g.time, " ", zg.zza[k], " ", z.zro[center_i, cente
r_j, k])
    end

    for i in 1:imax
        println(io_units[68], g.time, " ", zg.zxa[i], " ", z.zux[i, center_j, ce
nter_k])
    end

    for j in 1:jmax
        println(io_units[88], g.time, " ", zg.zya[j], " ", z.zuy[center_i, j, ce
nter_k])
    end

    for k in 1:kmax
        println(io_units[98], g.time, " ", zg.zza[k], " ", z.zuz[center_i, cente
r_j, k])
    end

    for i in 1:imax
        println(io_units[69], g.time, " ", zg.zxa[i], " ", z.zpr[i, center_j, ce
nter_k] / z.zro[i, center_j, center_k])
    end

    for i in 1:imax
        println(io_units[70], g.time, " ", zg.zxa[i], " ", z.zpr[i, center_j, ce
nter_k])
    end

    for i in 1:imax
        rloc = z.zro[i, center_j, center_k]
        ploc = z.zpr[i, center_j, center_k]
        sloc = slocal(rloc, ploc)
        println(io_units[71], g.time, " ", zg.zxa[i], " ", sloc, " ", sloc / rlo
c)
    end

    return nothing
end
```

```
# Julia port of ppmlr.f90

"""
    flatten0!(sweeps::Sweeps)

Set all flattening coefficients to zero.
"""
function flatten0!(sweeps::Sweeps)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    for n in (nmin - 4):(nmax + 4)
        sweeps.flat[n] = 0.0
    end
    return nothing
end

"""
    riemann0!(lmin, lmax, gamma, prgh, urgh, vrgh, plft, ulft, vlft, pmid, umid)

Simple Riemann average used by the Julia port.
"""
function riemann0!(lmin::Integer, lmax::Integer, gamma::Real,
                   prgh::AbstractVector{<:Real}, urgh::AbstractVector{<:Real},
                   vrgh::AbstractVector{<:Real}, plft::AbstractVector{<:Real},
                   ulft::AbstractVector{<:Real}, vlft::AbstractVector{<:Real},
                   pmid::AbstractVector{<:Real}, umid::AbstractVector{<:Real})
    smallp_local = 1.0e-25
    for l in lmin:lmax
        umid[l] = 0.5 * (urgh[l] + ulft[l])
        pmid[l] = 0.5 * (prgh[l] + plft[l])
        pmid[l] = max(smallp_local, pmid[l])
    end
    return nothing
end

"""
    volume!(sweeps::Sweeps)

Compute zone volumes and copy the current geometry into the remap arrays.
"""
function volume!(sweeps::Sweeps)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    sweeps.radius = 1.0
    for n in (nmin - 3):(nmax + 4)
        sweeps.dvol[n] = sweeps.dx[n]
        sweeps.dvol0[n] = sweeps.dx0[n]
    end
    return nothing
end

"""
    ppmlr!(sweeps, g, w)

Run the PPM/Lagrangian-remap workflow.
"""
function ppmlr!(sweeps::Sweeps, g::globalvars, w::PpmWork)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    para = w.para

    boundary!(sweeps)
    paraset!(para)
    volume!(sweeps)
    flatten0!(sweeps)
```

```
    dr = w.dr
    du = w.du
    dp = w.dp
    r6 = w.r6
    u6 = w.u6
    p6 = w.p6
    rl = w.rl
    ul = w.ul
    pl = w.pl
    rrgh = w.rrgh
    urgh = w.urgh
    prgh = w.prgh
    rlft = w.rlft
    ulft = w.ulft
    plft = w.plft
    umid = w.umid
    pmid = w.pmid

    parabola!(nmin - 4, nmax + 4, para, sweeps.p, dp, p6, pl, sweeps.flat, w)
    parabola!(nmin - 4, nmax + 4, para, sweeps.r, dr, r6, rl, sweeps.flat, w)
    parabola!(nmin - 4, nmax + 4, para, sweeps.u, du, u6, ul, sweeps.flat, w)

    states!(sweeps, g, pl, ul, rl, p6, u6, r6, dp, du, dr, plft, ulft, rlft, prg
h, urgh, rrgh, w)
    riemann0!(nmin - 3, nmax + 4, gam, prgh, urgh, rrgh, plft, ulft, rlft, pmid,
 umid)

    evolve!(sweeps, g, umid, pmid, w)
    remap!(sweeps, w)

    return nothing
end
```

```
# Julia port of boundary.f90

"""
    boundary!(sweeps::Sweeps)

Apply left and right boundary conditions to the 1D sweep arrays.
"""
function boundary!(sweeps::Sweeps)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    nleft = sweeps.nleft
    nright = sweeps.nright

    for n in 1:6
        if nleft == 0
            sweeps.dx[nmin - n] = sweeps.dx[nmin + n - 1]
            sweeps.xa[nmin - n] = sweeps.xa[nmin - n + 1] - sweeps.dx[nmin - n]
            sweeps.dx0[nmin - n] = sweeps.dx0[nmin + n - 1]
            sweeps.xa0[nmin - n] = sweeps.xa0[nmin - n + 1] - sweeps.dx0[nmin -
n]
            sweeps.r[nmin - n] = sweeps.r[nmin + n - 1]
            sweeps.u[nmin - n] = -sweeps.u[nmin + n - 1]
            sweeps.v[nmin - n] = sweeps.v[nmin + n - 1]
            sweeps.w[nmin - n] = sweeps.w[nmin + n - 1]
            sweeps.p[nmin - n] = sweeps.p[nmin + n - 1]
            sweeps.e[nmin - n] = sweeps.e[nmin + n - 1]
        elseif nleft == 1
            sweeps.dx[nmin - n] = sweeps.dx[nmin]
            sweeps.xa[nmin - n] = sweeps.xa[nmin - n + 1] - sweeps.dx[nmin - n]
            sweeps.dx0[nmin - n] = sweeps.dx0[nmin]
            sweeps.xa0[nmin - n] = sweeps.xa0[nmin - n + 1] - sweeps.dx0[nmin -
n]
            sweeps.r[nmin - n] = sweeps.r[nmin]
            sweeps.u[nmin - n] = sweeps.u[nmin]
            sweeps.v[nmin - n] = sweeps.v[nmin]
            sweeps.w[nmin - n] = sweeps.w[nmin]
            sweeps.p[nmin - n] = sweeps.p[nmin]
            sweeps.e[nmin - n] = sweeps.e[nmin]
        elseif nleft == 2
            sweeps.dx[nmin - n] = sweeps.dx[nmin]
            sweeps.xa[nmin - n] = sweeps.xa[nmin - n + 1] - sweeps.dx[nmin - n]
            sweeps.dx0[nmin - n] = sweeps.dx0[nmin]
            sweeps.xa0[nmin - n] = sweeps.xa0[nmin - n + 1] - sweeps.dx0[nmin -
n]
            sweeps.r[nmin - n] = dinflo
            sweeps.u[nmin - n] = uinflo
            sweeps.v[nmin - n] = vinflo
            sweeps.w[nmin - n] = winflo
            sweeps.p[nmin - n] = pinflo
            sweeps.e[nmin - n] = pinflo / (dinflo * gamm) + 0.5 * (uinflo^2 + vi
nflo^2 + winflo^2)
        elseif nleft == 3
            sweeps.dx[nmin - n] = sweeps.dx[nmax + 1 - n]
            sweeps.xa[nmin - n] = sweeps.xa[nmin - n + 1] - sweeps.dx[nmin - n]
            sweeps.dx0[nmin - n] = sweeps.dx0[nmax + 1 - n]
            sweeps.xa0[nmin - n] = sweeps.xa0[nmin - n + 1] - sweeps.dx0[nmin -
n]
            sweeps.r[nmin - n] = sweeps.r[nmax + 1 - n]
            sweeps.u[nmin - n] = sweeps.u[nmax + 1 - n]
            sweeps.v[nmin - n] = sweeps.v[nmax + 1 - n]
            sweeps.w[nmin - n] = sweeps.w[nmax + 1 - n]
            sweeps.p[nmin - n] = sweeps.p[nmax + 1 - n]
            sweeps.e[nmin - n] = sweeps.e[nmax + 1 - n]
        end
    end
```

```
    for n in 1:6
        if nright == 0
            sweeps.dx[nmax + n] = sweeps.dx[nmax + 1 - n]
            sweeps.xa[nmax + n] = sweeps.xa[nmax + n - 1] + sweeps.dx[nmax + n -
 1]
            sweeps.dx0[nmax + n] = sweeps.dx0[nmax + 1 - n]
            sweeps.xa0[nmax + n] = sweeps.xa0[nmax + n - 1] + sweeps.dx0[nmax +
n - 1]
            sweeps.r[nmax + n] = sweeps.r[nmax + 1 - n]
            sweeps.u[nmax + n] = -sweeps.u[nmax + 1 - n]
            sweeps.v[nmax + n] = sweeps.v[nmax + 1 - n]
            sweeps.w[nmax + n] = sweeps.w[nmax + 1 - n]
            sweeps.p[nmax + n] = sweeps.p[nmax + 1 - n]
            sweeps.e[nmax + n] = sweeps.e[nmax + 1 - n]
        elseif nright == 1
            sweeps.dx[nmax + n] = sweeps.dx[nmax]
            sweeps.xa[nmax + n] = sweeps.xa[nmax + n - 1] + sweeps.dx[nmax + n -
 1]
            sweeps.dx0[nmax + n] = sweeps.dx0[nmax]
            sweeps.xa0[nmax + n] = sweeps.xa0[nmax + n - 1] + sweeps.dx0[nmax +
n - 1]
            sweeps.r[nmax + n] = sweeps.r[nmax]
            sweeps.u[nmax + n] = sweeps.u[nmax]
            sweeps.v[nmax + n] = sweeps.v[nmax]
            sweeps.w[nmax + n] = sweeps.w[nmax]
            sweeps.p[nmax + n] = sweeps.p[nmax]
            sweeps.e[nmax + n] = sweeps.e[nmax]
        elseif nright == 2
            sweeps.dx[nmax + n] = sweeps.dx[nmax]
            sweeps.xa[nmax + n] = sweeps.xa[nmax + n - 1] + sweeps.dx[nmax + n -
 1]
            sweeps.dx0[nmax + n] = sweeps.dx0[nmax]
            sweeps.xa0[nmax + n] = sweeps.xa0[nmax + n - 1] + sweeps.dx0[nmax +
n - 1]
            sweeps.r[nmax + n] = dotflo
            sweeps.u[nmax + n] = uotflo
            sweeps.v[nmax + n] = votflo
            sweeps.w[nmax + n] = wotflo
            sweeps.p[nmax + n] = potflo
            sweeps.e[nmax + n] = potflo / (dotflo * gamm) + 0.5 * (uotflo^2 + vo
tflo^2 + wotflo^2)
        elseif nright == 3
            sweeps.dx[nmax + n] = sweeps.dx[nmin + n - 1]
            sweeps.xa[nmax + n] = sweeps.xa[nmax + n - 1] + sweeps.dx[nmax + n -
 1]
            sweeps.dx0[nmax + n] = sweeps.dx0[nmin + n - 1]
            sweeps.xa0[nmax + n] = sweeps.xa0[nmax + n - 1] + sweeps.dx0[nmax +
n - 1]
            sweeps.r[nmax + n] = sweeps.r[nmin + n - 1]
            sweeps.u[nmax + n] = sweeps.u[nmin + n - 1]
            sweeps.v[nmax + n] = sweeps.v[nmin + n - 1]
            sweeps.w[nmax + n] = sweeps.w[nmin + n - 1]
            sweeps.p[nmax + n] = sweeps.p[nmin + n - 1]
            sweeps.e[nmax + n] = sweeps.e[nmin + n - 1]
        end
    end

    return nothing
end
```

```
# Julia port of parabola.f90

function parabola!(nmin0::Integer, nmax0::Integer, para::Vector{Float64},
                   a::Vector{Float64}, deltaa::Vector{Float64},
                   a6::Vector{Float64}, al::Vector{Float64},
                   flat::Vector{Float64}, w::PpmWork)

    diffa = w.diffa
    da = w.da
    ar = w.ar
    scrch1 = w.scrch1
    scrch2 = w.scrch2
    scrch3 = w.scrch3

    for n in (nmin0-2):(nmax0 + 1)
        diffa[n] = a[n + 1] - a[n]
    end

    for n in (nmin0-1):(nmax0+1)
        da[n] = para[4] * diffa[n] + para[5] * diffa[n - 1]
        da[n] = copysign(min(abs(da[n]), 2.0 * abs(diffa[n - 1]), 2.0 * abs(diff
a[n])), da[n])
    end

    for n in (nmin0-1):(nmax0+1)
        if diffa[n - 1] * diffa[n] < 0.0
            da[n] = 0.0
        end
    end

    for n in (nmin0-1):nmax0
        ar[n] = a[n] + para[1] * diffa[n] + para[2] * da[n + 1] + para[3] * da[n
]
        al[n + 1] = ar[n]
    end

    for n in nmin0:nmax0
        onemfl = 1.0 - flat[n]
        ar[n] = flat[n] * a[n] + onemfl * ar[n]
        al[n] = flat[n] * a[n] + onemfl * al[n]
    end

    for n in nmin0:nmax0
        deltaa[n] = ar[n] - al[n]
        a6[n] = 6.0 * (a[n] - 0.5 * (al[n] + ar[n]))
        scrch1[n] = (ar[n] - a[n]) * (a[n] - al[n])
        scrch2[n] = deltaa[n] * deltaa[n]
        scrch3[n] = deltaa[n] * a6[n]
    end

    for n in nmin0:nmax0
        if scrch1[n] <= 0.0
            ar[n] = a[n]
            al[n] = a[n]
        end
        if scrch2[n] < scrch3[n]
            al[n] = 3.0 * a[n] - 2.0 * ar[n]
        end
        if scrch2[n] < -scrch3[n]
            ar[n] = 3.0 * a[n] - 2.0 * al[n]
        end
    end

    for n in nmin0:nmax0
        deltaa[n] = ar[n] - al[n]
        a6[n] = 6.0 * (a[n] - 0.5 * (al[n] + ar[n]))
```

```
    end

    return nothing
end

function paraset!(para::Vector{Float64})
    para[1] = 0.5
    para[2] = -1.0 / 6.0
    para[3] = 1.0 / 6.0
    para[4] = 0.5
    para[5] = 0.5
    return nothing
end
```

```
# Julia port of states.f90

"""
    states!(sweeps, g, pl, ul, rl, p6, u6, r6, dp, du, dr,
            plft, ulft, rlft, prgh, urgh, rrgh, w)

Compute characteristic left/right states from zone-edge variables and slopes.
"""
function states!(sweeps::Sweeps, g::globalvars,
                 pl::AbstractVector{<:Real}, ul::AbstractVector{<:Real}, rl::Abs
tractVector{<:Real},
                 p6::AbstractVector{<:Real}, u6::AbstractVector{<:Real}, r6::Abs
tractVector{<:Real},
                 dp::AbstractVector{<:Real}, du::AbstractVector{<:Real}, dr::Abs
tractVector{<:Real},
                 plft::AbstractVector{<:Real}, ulft::AbstractVector{<:Real}, rlf
t::AbstractVector{<:Real},
                 prgh::AbstractVector{<:Real}, urgh::AbstractVector{<:Real}, rrg
h::AbstractVector{<:Real},
                 w::PpmWork)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    fourthd = 4.0 / 3.0
    hdt = 0.5 * g.dt

    Cdtdx = w.Cdtdx
    fCdtdx = w.fCdtdx

    for n in (nmin - 4):(nmax + 4)
        Cdtdx[n] = sqrt(gam * sweeps.p[n] / sweeps.r[n]) / (sweeps.dx[n] * sweep
s.radius)
        Cdtdx[n] = Cdtdx[n] * hdt
        fCdtdx[n] = 1.0 - fourthd * Cdtdx[n]
    end

    for n in (nmin - 4):(nmax + 4)
        np = n + 1
        plft[np] = pl[n] + dp[n] - Cdtdx[n] * (dp[n] - fCdtdx[n] * p6[n])
        ulft[np] = ul[n] + du[n] - Cdtdx[n] * (du[n] - fCdtdx[n] * u6[n])
        rlft[np] = rl[n] + dr[n] - Cdtdx[n] * (dr[n] - fCdtdx[n] * r6[n])
        plft[np] = max(smallp, plft[np])
        rlft[np] = max(smallr, rlft[np])

        prgh[n] = pl[n] + Cdtdx[n] * (dp[n] + fCdtdx[n] * p6[n])
        urgh[n] = ul[n] + Cdtdx[n] * (du[n] + fCdtdx[n] * u6[n])
        rrgh[n] = rl[n] + Cdtdx[n] * (dr[n] + fCdtdx[n] * r6[n])
        prgh[n] = max(smallp, prgh[n])
        rrgh[n] = max(smallr, rrgh[n])
    end

    return nothing
end
```

```
# Julia port of evolve.f90

"""
    evolve!(sweeps, g, umid, pmid, w)

Lagrangian update for density, velocity, and total energy.
"""
function evolve!(sweeps::Sweeps, g::globalvars,
                 umid::AbstractVector{<:Real}, pmid::AbstractVector{<:Real},
                 w::PpmWork)
    nmin = sweeps.nmin
    nmax = sweeps.nmax

    amid = w.amid
    uold = w.uold
    xa1 = w.xa1
    dvol1 = w.dvol1
    upmid = w.upmid
    dm = w.dm
    dtbdm = w.dtbdm
    xa2 = w.xa2
    xa3 = w.xa3

    for n in (nmin - 3):(nmax + 4)
        dm[n]    = sweeps.r[n] * sweeps.dvol[n]
        dtbdm[n] = g.dt / dm[n]
        xa1[n]   = sweeps.xa[n]
        dvol1[n] = sweeps.dvol[n]
        sweeps.xa[n]    = sweeps.xa[n] + g.dt * umid[n] / sweeps.radius
        upmid[n] = umid[n] * pmid[n]
    end

    xa1[nmin - 4] = sweeps.xa[nmin - 4]
    xa1[nmax + 5] = sweeps.xa[nmax + 5]

    for n in (nmin - 4):(nmax + 5)
        xa2[n] = xa1[n] + 0.5 * sweeps.dx[n]
        sweeps.dx[n]  = sweeps.xa[n + 1] - sweeps.xa[n]
        xa3[n] = sweeps.xa[n] + 0.5 * sweeps.dx[n]
    end

    for n in (nmin - 3):(nmax + 4)
        sweeps.dvol[n] = sweeps.dx[n]
        amid[n] = 1.0
    end

    for n in (nmin - 3):(nmax + 3)
        sweeps.r[n] = sweeps.r[n] * (dvol1[n] / sweeps.dvol[n])
        sweeps.r[n] = max(sweeps.r[n], smallr)

        uold[n] = sweeps.u[n]
        sweeps.u[n] = sweeps.u[n] - dtbdm[n] * (pmid[n + 1] - pmid[n]) * 0.5 * (
amid[n + 1] + amid[n])

        sweeps.e[n] = sweeps.e[n] - dtbdm[n] * (amid[n + 1] * upmid[n + 1] - ami
d[n] * upmid[n])
        sweeps.q[n] = sweeps.e[n] - 0.5 * (sweeps.u[n]^2 + sweeps.v[n]^2 + sweep
s.w[n]^2)
        sweeps.p[n] = max(sweeps.r[n] * sweeps.q[n] * gamm, smallp)
    end

    return nothing
end
```

```
# Julia port of remap.f90

"""
    remap!(sweeps, w)

Apply the remap update from the Lagrangian grid to the Eulerian grid.
"""
function remap!(sweeps::Sweeps, w::PpmWork)
    nmin = sweeps.nmin
    nmax = sweeps.nmax
    fourthd = 4.0 / 3.0

    para = w.para
    du = w.du
    ul = w.ul
    u6 = w.u6
    dv = w.dv
    vl = w.vl
    v6 = w.v6
    dw = w.dw
    wl = w.wl
    w6 = w.w6
    de = w.de
    el = w.el
    e6 = w.e6
    dq = w.dq
    ql = w.ql
    q6 = w.q6
    dr = w.dr
    rl = w.rl
    r6 = w.r6
    dm = w.dm
    dm0 = w.dm0
    delta = w.delta
    fluxr = w.fluxr
    fluxu = w.fluxu
    fluxv = w.fluxv
    fluxw = w.fluxw
    fluxe = w.fluxe
    fluxq = w.fluxq

    paraset!(para)
    parabola!(nmin - 1, nmax + 1, para, sweeps.r, dr, r6, rl, sweeps.flat, w)
    parabola!(nmin - 1, nmax + 1, para, sweeps.u, du, u6, ul, sweeps.flat, w)
    parabola!(nmin - 1, nmax + 1, para, sweeps.v, dv, v6, vl, sweeps.flat, w)
    parabola!(nmin - 1, nmax + 1, para, sweeps.w, dw, w6, wl, sweeps.flat, w)
    parabola!(nmin - 1, nmax + 1, para, sweeps.q, dq, q6, ql, sweeps.flat, w)
    parabola!(nmin - 1, nmax + 1, para, sweeps.e, de, e6, el, sweeps.flat, w)

    for n in nmin:(nmax + 1)
        delta[n] = sweeps.xa[n] - sweeps.xa0[n]
    end

    for n in nmin:(nmax + 1)
        deltx = sweeps.xa[n] - sweeps.xa0[n]
        if deltx >= 0.0
            nn = n - 1
            fractn = 0.5 * deltx / sweeps.dx[nn]
            fractn2 = 1.0 - fourthd * fractn
            fluxr[n] = (rl[nn] + dr[nn] - fractn * (dr[nn] - fractn2 * r6[nn]))
* delta[n]
            fluxu[n] = (ul[nn] + du[nn] - fractn * (du[nn] - fractn2 * u6[nn]))
* fluxr[n]
            fluxv[n] = (vl[nn] + dv[nn] - fractn * (dv[nn] - fractn2 * v6[nn]))
* fluxr[n]
            fluxw[n] = (wl[nn] + dw[nn] - fractn * (dw[nn] - fractn2 * w6[nn]))
```

```
* fluxr[n]
            fluxe[n] = (el[nn] + de[nn] - fractn * (de[nn] - fractn2 * e6[nn]))
* fluxr[n]
            fluxq[n] = (ql[nn] + dq[nn] - fractn * (dq[nn] - fractn2 * q6[nn]))
* fluxr[n]
        else
            fractn = 0.5 * deltx / sweeps.dx[n]
            fractn2 = 1.0 + fourthd * fractn
            fluxr[n] = (rl[n] - fractn * (dr[n] + fractn2 * r6[n])) * delta[n]
            fluxu[n] = (ul[n] - fractn * (du[n] + fractn2 * u6[n])) * fluxr[n]
            fluxv[n] = (vl[n] - fractn * (dv[n] + fractn2 * v6[n])) * fluxr[n]
            fluxw[n] = (wl[n] - fractn * (dw[n] + fractn2 * w6[n])) * fluxr[n]
            fluxe[n] = (el[n] - fractn * (de[n] + fractn2 * e6[n])) * fluxr[n]
            fluxq[n] = (ql[n] - fractn * (dq[n] + fractn2 * q6[n])) * fluxr[n]
        end
    end

    for n in nmin:nmax
        dm[n] = sweeps.r[n] * sweeps.dvol[n]
        dm0[n] = dm[n] + fluxr[n] - fluxr[n + 1]
        sweeps.r[n] = dm0[n] / sweeps.dvol0[n]
        sweeps.r[n] = max(smallr, sweeps.r[n])
        dm0[n] = 1.0 / (sweeps.r[n] * sweeps.dvol0[n])
        sweeps.u[n] = (sweeps.u[n] * dm[n] + fluxu[n] - fluxu[n + 1]) * dm0[n]
        sweeps.v[n] = (sweeps.v[n] * dm[n] + fluxv[n] - fluxv[n + 1]) * dm0[n]
        sweeps.w[n] = (sweeps.w[n] * dm[n] + fluxw[n] - fluxw[n + 1]) * dm0[n]
        sweeps.e[n] = (sweeps.e[n] * dm[n] + fluxe[n] - fluxe[n + 1]) * dm0[n]
        sweeps.q[n] = (sweeps.q[n] * dm[n] + fluxq[n] - fluxq[n + 1]) * dm0[n]

        ekin = 0.5 * (sweeps.u[n]^2 + sweeps.v[n]^2 + sweeps.w[n]^2)
        if ekin / sweeps.q[n] < 100.0
            sweeps.q[n] = sweeps.e[n] - ekin
        end

        sweeps.p[n] = gamm * sweeps.r[n] * sweeps.q[n]
        sweeps.p[n] = max(smallp, sweeps.p[n])
    end

    return nothing
end
```

```
# Constants and state types for the VH-1 Julia port.

using Random

const maxsweep = 1036

const imax = 40
const jmax = 40
const kmax = 40
const ndim = 3

const smallp = 1.0e-15
const smallr = 1.0e-15
const small = 1.0e-15

const pi = 2.0 * asin(1.0)
const gam = 5.0 / 3.0
const gamm = gam - 1.0
const alpha = 0.0
const alphat = 0.0
const nav = 400.0
const tav = 1.0
const etaav = 0.1 * nav

const A0 = 0.25 * 0.25
const V0 = A0 * 0.25

const uinflo = 0.0
const dinflo = 0.0
const vinflo = 0.0
const winflo = 0.0
const pinflo = 0.0
const einflo = 0.0

const uotflo = 0.0
const dotflo = 0.0
const votflo = 0.0
const wotflo = 0.0
const potflo = 0.0
const eotflo = 0.0

mutable struct globalvars
    ncycle::Int
    ncycp::Int
    ntrial::Int
    nacc::Int
    time::Float64
    dt::Float64
    timep::Float64
    dtfix::Float64
    endtime::Float64
    tprin::Float64
end

function globalvars()
    globalvars(0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
end

mutable struct Sweeps
    nmin::Int
    nmax::Int
    nleft::Int
    nright::Int
    r::Vector{Float64}
    p::Vector{Float64}
    e::Vector{Float64}
```

```
    q::Vector{Float64}
    u::Vector{Float64}
    v::Vector{Float64}
    w::Vector{Float64}
    xa::Vector{Float64}
    xa0::Vector{Float64}
    dx::Vector{Float64}
    dx0::Vector{Float64}
    dvol::Vector{Float64}
    dvol0::Vector{Float64}
    flat::Vector{Float64}
    radius::Float64
    area::Float64
end

function Sweeps()
    Sweeps(
        0, 0, 0, 0,
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        zeros(Float64, maxsweep),
        0.0, 0.0
    )
end

mutable struct zone
    zro::Array{Float64,3}
    zpr::Array{Float64,3}
    zux::Array{Float64,3}
    zuy::Array{Float64,3}
    zuz::Array{Float64,3}
end

function zone()
    zone(
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax)
    )
end

mutable struct zonegrid
    zxa::Vector{Float64}
    zdx::Vector{Float64}
    zxc::Vector{Float64}
    zya::Vector{Float64}
    zdy::Vector{Float64}
    zyc::Vector{Float64}
    zza::Vector{Float64}
    zdz::Vector{Float64}
    zzc::Vector{Float64}
    nleftx::Int
    nlefty::Int
```

```
    nleftz::Int
    nrightx::Int
    nrighty::Int
    nrightz::Int
end

function zonegrid()
    zonegrid(
        zeros(Float64, imax),
        zeros(Float64, imax),
        zeros(Float64, imax),
        zeros(Float64, jmax),
        zeros(Float64, jmax),
        zeros(Float64, jmax),
        zeros(Float64, kmax),
        zeros(Float64, kmax),
        zeros(Float64, kmax),
        0, 0, 0, 0, 0, 0
    )
end

mutable struct zoneobs
    repixky::Vector{Float64}
    impixky::Vector{Float64}
    spec::Vector{Float64}
    spec2::Vector{Float64}
    rhokx::Vector{Float64}
end

function zoneobs()
    zoneobs(
        zeros(Float64, jmax),
        zeros(Float64, jmax),
        zeros(Float64, jmax),
        zeros(Float64, jmax),
        zeros(Float64, imax)
    )
end

# Reusable scratch buffers for ppmlr / parabola / remap / evolve / states.
mutable struct PpmWork
    para::Vector{Float64}
    diffa::Vector{Float64}
    da::Vector{Float64}
    ar::Vector{Float64}
    scrch1::Vector{Float64}
    scrch2::Vector{Float64}
    scrch3::Vector{Float64}
    dr::Vector{Float64}
    du::Vector{Float64}
    dp::Vector{Float64}
    r6::Vector{Float64}
    u6::Vector{Float64}
    p6::Vector{Float64}
    rl::Vector{Float64}
    ul::Vector{Float64}
    pl::Vector{Float64}
    rrgh::Vector{Float64}
    urgh::Vector{Float64}
    prgh::Vector{Float64}
    rlft::Vector{Float64}
    ulft::Vector{Float64}
    plft::Vector{Float64}
    umid::Vector{Float64}
    pmid::Vector{Float64}
    Cdtdx::Vector{Float64}
```

```
    fCdtdx::Vector{Float64}
    amid::Vector{Float64}
    uold::Vector{Float64}
    xa1::Vector{Float64}
    dvol1::Vector{Float64}
    upmid::Vector{Float64}
    dm::Vector{Float64}
    dtbdm::Vector{Float64}
    xa2::Vector{Float64}
    xa3::Vector{Float64}
    dv::Vector{Float64}
    vl::Vector{Float64}
    v6::Vector{Float64}
    dw::Vector{Float64}
    wl::Vector{Float64}
    w6::Vector{Float64}
    de::Vector{Float64}
    el::Vector{Float64}
    e6::Vector{Float64}
    dq::Vector{Float64}
    ql::Vector{Float64}
    q6::Vector{Float64}
    dm0::Vector{Float64}
    delta::Vector{Float64}
    fluxr::Vector{Float64}
    fluxu::Vector{Float64}
    fluxv::Vector{Float64}
    fluxw::Vector{Float64}
    fluxe::Vector{Float64}
    fluxq::Vector{Float64}
end

function _ppm_vec()
    zeros(Float64, maxsweep)
end

function PpmWork()
    PpmWork(
        zeros(Float64, 5),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(),
        _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec(), _ppm_vec()
    )
end

# Shared grid buffers for mc3d (read/written by non-overlapping cell sets per co
lor phase).
mutable struct McWork
    p1::Array{Float64,3}
    p2::Array{Float64,3}
    p3::Array{Float64,3}
    q::Array{Float64,3}
end

function McWork()
    McWork(
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax),
        zeros(Float64, imax, jmax, kmax)
```

```
    )
end

# Per-thread scratch + RNG for parallel mc3d (no per-cell allocation).
mutable struct McScratch
    lambda::Matrix{Float64}
    lambda_t::Matrix{Float64}
    delta_p::Matrix{Float64}
    qnew_cube::Vector{Float64}
    pcube1::Vector{Float64}
    pcube2::Vector{Float64}
    pcube3::Vector{Float64}
    rng::Xoshiro
end

function McScratch(tid::Int)
    McScratch(
        zeros(Float64, 3, 3),
        zeros(Float64, 3, 3),
        zeros(Float64, 3, 3),
        zeros(Float64, 8),
        zeros(Float64, 8),
        zeros(Float64, 8),
        zeros(Float64, 8),
        Xoshiro(123456 + tid * 100003)
    )
end

mutable struct McThreadPool
    grid::McWork
    scratch::Vector{McScratch}
end

function McThreadPool()
    n = Threads.maxthreadid()
    McThreadPool(McWork(), [McScratch(tid) for tid in 1:n])
end

@inline mc_phase_count(n::Int, parity::Int) = (n - parity + 1) Ã• 2

# Per-thread Sweeps + PpmWork for parallel line sweeps (sweepx/y/z).
mutable struct SweepThreadPool
    sweeps::Vector{Sweeps}
    work::Vector{PpmWork}
end

function SweepThreadPool()
    n = Threads.maxthreadid()
    SweepThreadPool([Sweeps() for _ in 1:n], [PpmWork() for _ in 1:n])
end

function _build_pixky_trig()
    nk = jmax
    cos_j = Matrix{Float64}(undef, jmax, nk)
    sin_j = Matrix{Float64}(undef, jmax, nk)
    sin_i = Matrix{Float64}(undef, imax, nk)
    for in_ in 1:nk
        for j in 1:jmax
            ang = 2.0 * ÏM-^@ * j * in_ / jmax
            cos_j[j, in_] = cos(ang)
            sin_j[j, in_] = sin(ang)
        end
        for i in 1:imax
            sin_i[i, in_] = sin(2.0 * ÏM-^@ * (i - 1) * in_ / imax)
        end
    end
```

```
    return cos_j, sin_j, sin_i
end

const PIXKY_COS_J, PIXKY_SIN_J, PIXKY_SIN_I = _build_pixky_trig()
const PIXKY_XNORM = 1.0 / sqrt(Float64(imax * jmax * kmax))

const MC_SIGN_I = (-1.0, 1.0, -1.0, 1.0, -1.0, 1.0, -1.0, 1.0)
const MC_SIGN_J = (-1.0, -1.0, 1.0, 1.0, -1.0, -1.0, 1.0, 1.0)
const MC_SIGN_K = (-1.0, -1.0, -1.0, -1.0, 1.0, 1.0, 1.0, 1.0)
const MC_I_SLOT = (1, 2, 1, 2, 1, 2, 1, 2)
const MC_J_SLOT = (1, 1, 2, 2, 1, 1, 2, 2)
const MC_K_SLOT = (1, 1, 1, 1, 2, 2, 2, 2)
```